Adaptive Decision Intelligence for Crisis management

A Classical AI-Based Decision Support System Combining Rule-Based Reasoning, Heuristic Search, and Constraint Satisfaction for Emergency Response Planning

Akshaya D.

Department of Computer Science and Engineering (Cybersecurity)

Rajalakshmi Engineering College Chennai, India

akshaya.d.2024.csecs@rajalakshmi.edu.in

Angelin Praisee K.

Department of Computer Science and Engineering (Cybersecurity)

Rajalakshmi Engineering College

Chennai, India

angelinpraisee.k.2024.csecs@rajalakshmi.edu.in

Abstract - Emergency response planning involves prioritizing incidents, matching scarce resources to operational needs, estimating routes, and adapting decisions as conditions change. These tasks are challenging when incident severity, resource availability, road access, and communication conditions evolve at different rates. This paper presents Adaptive Decision Intelligence for Crisis Management, a modular decision-support prototype intended to assist an emergency coordinator in reviewing response options. The proposed workflow combines incident and location management, weighted priority assessment, resource-candidate generation, constraint-based filtering, A*-based route search when suitable road-network data are available, multi-factor resource scoring, explanation, and event-driven re-planning. A Python and Streamlit implementation provides a dashboard through which incident inputs, resource status, route details, and revised plans can be inspected. The system emphasizes feasibility checks and interpretable recommendations rather than opaque automated decisions. A changed condition can trigger re-evaluation and a comparison between the previous and updated plans. This paper describes the motivation, architecture, algorithmic workflow, implementation considerations, and a reproducible evaluation plan. No measured field-performance improvement is claimed; empirical validation with realistic data and expert review remains future work.

Index Terms—crisis management, emergency response, decision support, constraint satisfaction, A* search, resource allocation, adaptive planning.

I. INTRODUCTION

Disasters and large-scale emergencies often produce multiple needs at the same time. A coordinator may need to compare incident severity, estimate the number of people affected, identify available response teams, account for travel restrictions, and decide whether an existing plan remains appropriate. The challenge is not simply finding a resource; it is selecting a feasible combination of resources while preserving a clear explanation of why the combination is preferred.

Disaster operations research has investigated resource allocation, logistics, facility location, and emergency dispatch under uncertainty [1], [2]. Decision-support systems can bring data and analytical methods together so that users can examine alternatives rather than relying only on informal judgement [3]. However, the value of a decision-support tool depends on data quality, understandable constraints, and whether its output can be reviewed by the people responsible for the response.

This work presents Adaptive Decision Intelligence for Crisis Management (ADICM), a modular prototype for emergency-response planning. It is designed to accept incident details and resource information, calculate a configurable priority score, generate candidate assignments, reject combinations that violate hard constraints, estimate routes when network data are available, and present a response plan. The prototype also supports event-driven re-planning: after a road closure, resource loss, or other change is introduced, the system updates the state and compares the new plan with the previous one.

The contribution is an integrated, operator-facing workflow rather than a new search algorithm. Its main design objectives are: (1) modular separation of incident, resource, route, and allocation logic; (2) explicit hard-constraint checks; (3) explanation of plan choices and infeasibility; and (4) a repeatable mechanism for re-evaluating decisions after a change. The intended user is an emergency coordinator who retains responsibility for interpreting and approving recommendations.

II. PROBLEM STATEMENT AND OBJECTIVES

Let an incident be represented by a profile containing its type, severity, estimated affected population, location, time, and contextual conditions. Let the resource set contain response units with attributes such as capability, availability, capacity,

current workload, and location. The planning problem is to identify an assignment of available resources to incident needs that respects hard constraints while balancing urgency, suitability, and estimated travel cost.

The problem is dynamic because both incident conditions and resource states may change after a plan is generated. A route may become inaccessible, a unit may be reassigned, or the severity estimate may be updated. A static allocation can therefore become invalid. ADICM treats the response plan as a versioned decision that can be re-evaluated when a new event is introduced.

The project has five objectives. First, it provides a structured input workflow and validates incident data. Second, it calculates an explainable priority indicator from configured factors. Third, it narrows the resource search space by matching capabilities and applying hard constraints. Fourth, it uses route search and option scoring to help rank feasible alternatives. Fifth, it records the rationale for a plan and supports comparison after changes. These objectives focus on transparency and operational feasibility; they do not imply that the system can replace emergency command or guarantee the safety of a route.

III. RELATED WORK

A. Decision Support for Disaster Management

Power describes decision-support systems as computer-based approaches that help users work with information and models in semi-structured decision situations [3]. In emergency response, information can arrive from multiple sources and can change while decisions are being made. Turoff et al. presented the design of a dynamic emergency response management information system, highlighting the importance of information management in evolving incidents [4]. These works motivate a system that connects operational data to a repeatable decision workflow.

Altay and Green surveyed operations research and management science research in disaster operations, while Galindo and Batta reviewed later developments in the area [1], [2]. The literature includes problems involving preparedness, response logistics, resource distribution, and recovery. ADICM addresses a related but narrower prototype-level task: bringing incident priority, resource feasibility, route estimation, and plan revision into one operator-facing process.

B. Search and Constraint-Based Planning

Hart, Nilsson, and Raphael established a formal basis for A* heuristic search for minimum-cost paths [5]. A* evaluates partial paths by combining the cost already incurred with a heuristic estimate of the remaining cost. Its practical quality depends on the graph representation, edge costs, and heuristic properties. For crisis response, a route estimate should therefore be presented with its data limitations; a shortest path on an outdated road graph cannot be interpreted as a guarantee of real-time safety.

Constraint satisfaction provides a way to represent requirements that must be met by a candidate solution. Mackworth's work on consistency in networks of relations is a foundational contribution to constraint processing [6]. In a resource allocation context, constraints may encode capability, availability, workload limits, travel reachability, and time windows. Applying these checks before ranking reduces the risk that a high-scoring but infeasible assignment is selected.

C. Multi-Criteria Decision Making

Keeney and Raiffa discuss decision analysis with multiple objectives and value trade-offs [7]. Crisis planning also involves competing objectives: responding quickly, meeting the most urgent needs, using appropriate resources, and avoiding excessive workload on a small subset of units. A weighted scoring model can make these trade-offs explicit, provided that the factors and weights are documented. The proposed system uses configurable scoring as a transparent decision aid, not as an objective truth about incident importance.

The reviewed approaches solve complementary subproblems. The design presented here connects them in a single sequence and adds a re-planning step that compares a previous plan with a plan generated after an event. Its novelty claim is therefore architectural and workflow-oriented, and must be evaluated through implementation and scenario testing rather than inferred from the use of known algorithms.

IV. SYSTEM ARCHITECTURE

The architecture separates the dashboard from decision services and algorithmic components. The presentation layer collects incident information and displays summaries, resource status, route information, and plan differences. The service layer coordinates incident management, resource management, explanation, and adaptation. The algorithm layer performs weighted scoring, constraint checks, route search, and allocation. A data layer stores incident records, resource attributes, road or location data, and test scenarios. External geocoding and map services may be connected when required.

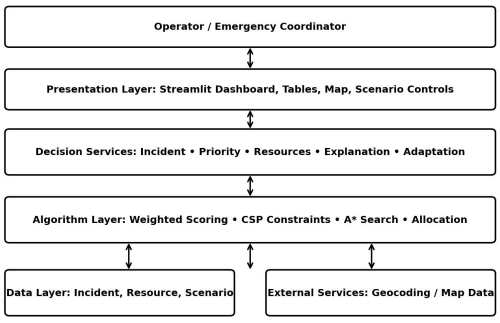


Fig. 1. High-level architecture of the proposed decision-support system.

A modular structure makes it easier to test a component independently. For example, the priority calculation can be checked using fixed inputs, constraint filtering can be tested against known valid and invalid assignments, and route search can be checked against a small graph with known paths. The interface should also distinguish actual data from simulated scenario values. If an external service is unavailable, the system should report the status rather than silently substituting an unverified result.

V. PROPOSED METHODOLOGY

A. Incident and Location Management

The workflow begins with incident type, severity, affected population, timestamp, location, and contextual fields such as weather, communication availability, and infrastructure status. Input validation checks required fields, supported categories, and numeric ranges. When geocoding is available, a location name is resolved to coordinates; otherwise, the location should be marked unresolved and route-dependent functions should not imply a precise result. A validated incident profile becomes the common input for subsequent modules.

B. Priority Assessment

The priority module converts configured incident factors into normalized values and combines them into a score. A generic weighted formulation is $P = 100 \times \Sigma(w_i x_i)$, subject to $\Sigma w_i = 1$, where x_i is a normalized factor in [0,1] and w_i is its configured weight. Example factors include severity, affected population, weather risk, infrastructure disruption, and communication availability. Factors that increase urgency should be oriented consistently before aggregation.

The score is intended to support comparison, not to dictate an emergency decision. The output should display the score and, where practical, the factors that contributed to it. Weights and thresholds require calibration with domain experts and should be reviewed for unintended bias. If a factor is missing, the implementation must specify whether it is rejected, imputed, or omitted with a warning; silently treating missing values as safe conditions would be inappropriate.

C. Resource and Candidate Generation

The candidate-generation module interprets incident needs and compares them with the capabilities of the resource inventory. For example, a need may require medical support, rescue capability, transport capacity, or logistics support. Resources that are unavailable or lack the required capability should not be included in the candidate set. Candidate records should retain the reason for inclusion so that later stages can trace how an option was formed.

This stage is a narrowing step rather than the final allocation. A resource may match a need but still be unusable because of workload, travel distance, time windows, or a conflict with another assignment. These additional conditions are checked by the constraint and route modules.

D. Constraint Filtering Using CSP Principles

Constraint filtering checks each candidate against operational restrictions. The prototype can represent resource availability, capability compatibility, workload limits, reachability, and deployment time windows as hard constraints. A candidate that violates a hard constraint is rejected before scoring. The result is a set of feasible candidates plus a record of failed checks. If all candidates are rejected, the system should state that no feasible assignment was found and allow the operator to review inputs or resource availability.

The distinction between hard and soft conditions is important. Hard constraints determine whether an assignment is allowed under the current model. Soft objectives help rank alternatives that remain feasible. A high priority score should not override an unavailable resource or an impossible route. Constraint rules must be visible and documented so that users can understand why a candidate was removed.

E. Route Intelligence with A*

When a road network is available as a graph G=(V,E), the route module searches for a path between a resource location and an incident location. A* evaluates each node n using $f(n)=g(n)+h(n)$, where g(n) is the accumulated path cost and h(n) estimates the remaining cost to the goal [5]. Edge costs may represent distance or a travel-time estimate. The heuristic should be compatible with the chosen cost model.

The module returns the route status, path where available, and an estimated distance or arrival time derived from the available data. If the graph is incomplete or no path is found, the route is marked unavailable. The prototype must not imply that a route estimate includes live traffic, road closures, weather hazards, or local emergency restrictions unless those inputs are actually connected and updated.

F. Scoring and Resource Allocation

After feasibility and route checks, the allocation module compares candidates using configured factors such as incident priority, capability match, travel time, and remaining resource capacity. One design is to first produce a feasible core assignment that covers the most important needs under hard constraints, then evaluate additional assignments by their marginal contribution to unmet needs. This staged approach makes it possible to distinguish essential coverage from optional improvements.

The exact scoring rule must be documented in the implementation. For instance, travel time may be normalized so that shorter estimated times receive higher preference, while capability match and need coverage may receive positive weights. Duplicate assignment, capacity overrun, and resource conflicts must remain prohibited regardless of score. The output is a response plan that lists selected resources, covered needs, route status, and any unmet demand.

G. Explanation and Adaptive Re-Planning

The explanation module summarizes the priority score, selected resources, estimated route details, and important reasons for the final assignment. It should also identify rejected candidates and the constraint that excluded them when that information is available. If no feasible plan exists, the system should present a limitation message rather than displaying an empty or misleading success state.

Adaptation begins when a new event or changed condition is introduced. The event updates the incident or resource state, after which the relevant decision stages are executed again. The system compares the new plan with the previous plan and reports additions, removals, route changes, and the reasons for those differences. Re-planning is especially useful for demonstrating that the application is not limited to a one-time calculation. It remains a decision aid: operators must verify changed information and approve any action.

VI. END-TO-END WORKFLOW

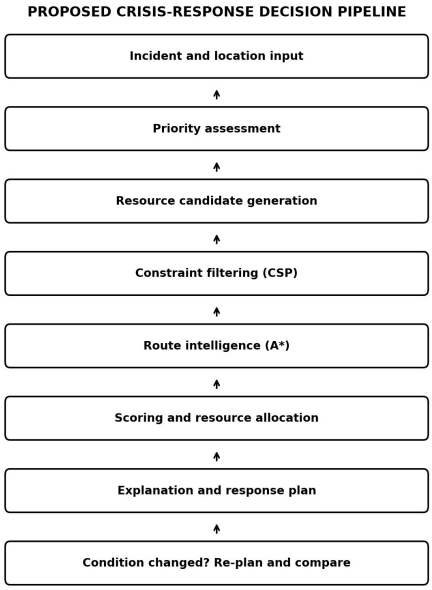


Fig. 2. Main processing sequence and event-driven re-planning loop.

The processing sequence starts by validating the incident and location. Priority assessment then establishes an urgency indicator, and candidate generation identifies resources that may meet the incident needs. Constraint filtering removes options that violate hard restrictions. Route intelligence estimates travel costs for remaining candidates when the required graph is available. Allocation ranks feasible options and constructs a plan. The explanation layer displays the result to the operator.

If the operator introduces a new event, the system updates its state and runs the affected stages again. Comparing old and new plans helps identify which recommendations changed and why. This loop is intended to support scenario analysis and sensitivity testing. In a production setting, every state change should be time-stamped and logged, and the system should retain the data source and version used for each recommendation.

VII. IMPLEMENTATION CONSIDERATIONS

A. Dashboard and Data Handling

Python provides the application logic, while Streamlit can expose inputs, tables, metrics, and visualizations through a browser-based interface. The implementation can organize incident, resource, and scenario data in structured records and separate business logic into functions or modules. Such separation helps reduce coupling between the interface and the decision algorithms. If a database is used, records should have stable identifiers and explicit timestamps so that plan comparisons can be reproduced.

The dashboard should show key information without requiring the user to inspect raw data. Useful displays include the priority score, required needs, eligible resources, rejected candidates and reasons, route status, allocation results, and differences after re-planning. A CSV export may support offline review, but exported files should include scenario metadata and a note distinguishing simulated values from externally sourced values.

B. External Services and Failure Handling

Geocoding and map tiles can improve location entry and visualization, but their availability and licensing conditions vary. External service failures should not crash the entire decision workflow. Instead, the application should show a clear warning, preserve the entered incident information, and disable functions that require missing coordinates or road-network data. A map is a visualization layer; it should not be treated as a certified emergency routing service.

C. Security, Reliability, and Human Oversight

Because the system handles incident and resource information, implementation should validate all user inputs, restrict access to sensitive operational records, avoid exposing secrets in source code, and log meaningful state changes. If deployed beyond a classroom prototype, authentication, authorization, backup, data retention, and audit requirements should be defined. The user interface should communicate uncertainty and data freshness, and it should make clear that recommendations require human review.

The application should fail safely. Invalid inputs should be rejected with a useful message; a missing location should not be converted into a fabricated route; and a lack of feasible assignments should be reported explicitly. These behaviors are as important as producing a plan under ideal conditions.

VIII. EVALUATION PLAN

This paper describes a prototype design and workflow and does not claim measured performance improvements. A reproducible evaluation should compare the system with a simple baseline, such as selecting the first available resource that matches a need, or a manually specified rule-based policy. The same incident scenarios and resource inventories should be used for both approaches.

A scenario suite should include: (1) a low-severity incident with ample resources; (2) a high-severity incident with limited resources; (3) a resource that matches capability but violates workload limits; (4) an incident with an unresolved location; (5) a disconnected or blocked route in the road graph; (6) simultaneous incidents competing for the same unit; and (7) a changed condition such as a road closure or resource loss. Each case should define expected constraints and the conditions under which a plan is considered feasible.

Recommended quantitative metrics include constraint-violation count, percentage of needs covered, total estimated travel time, number of unassigned needs, computation time, and number of assignments changed after an event. Route quality should be measured only against a known reference graph or verified route data. Qualitative evaluation can ask users whether the explanations are understandable, whether uncertainty is communicated, and whether the system makes it easier to compare options.

Results should be reported with scenario definitions, sample counts, assumptions, and limitations. If the system is tested only on synthetic data, that fact should be stated. A small set of demonstration scenarios can verify basic functionality but

cannot establish real-world effectiveness or readiness for emergency deployment.

IX. DISCUSSION

The main benefit of the proposed design is the connection between feasibility checking and explanation. A recommendation is more useful when the operator can see not only which resource was selected, but also which constraints were checked and why alternatives were rejected. The adaptation loop adds a second benefit: it provides a structured way to inspect how a decision changes after a new event. These properties can improve traceability in a prototype and make its behavior easier to test.

There are important limitations. First, priority scores depend on the factors and weights chosen by developers. Second, resource inventories may be incomplete or out of date. Third, route search can only be as reliable as the graph and edge costs supplied to it. Fourth, a simplified allocation model may not capture all real-world operational rules, such as crew rest, jurisdiction, equipment compatibility, or changing road conditions. Fifth, a dashboard can create false confidence if it displays precise-looking scores without explaining uncertainty.

For these reasons, the system should be evaluated as a decision-support prototype. Real deployment would require stakeholder engagement, data governance, operational validation, resilience testing, and a clear process for human approval. The system should not autonomously dispatch emergency resources based solely on a computed score.

X. CONCLUSION AND FUTURE WORK

This paper presented Adaptive Decision Intelligence for Crisis Management, a modular prototype that connects incident assessment, resource candidate generation, constraint filtering, route search, allocation, explanation, and event-driven re-planning. The design aims to help an emergency coordinator review feasible options, understand the reasons behind a recommendation, and compare plans after conditions change. It emphasizes transparent constraints and explicit failure states rather than presenting every scenario as solvable.

Future work will focus on implementing and validating the complete workflow against realistic and reproducible scenarios, connecting verified data sources, improving travel-time estimates, calibrating priority weights with domain experts, and evaluating explanation quality with users. Further extensions may include conflict resolution across multiple incidents, uncertainty-aware scoring, audit trails, and role-based access. Any claim of improved response time or allocation quality should be made only after suitable empirical evaluation.

XI. MODULE-LEVEL DESIGN

Incident and Location Management collects incident type, severity, affected population estimate, timestamp, contextual conditions, and location. Validation checks required fields, accepted categories, and numeric bounds. The system should preserve the original location text and any resolved coordinates, along with a status indicating whether geocoding succeeded. This helps distinguish a confirmed coordinate from a location that needs operator attention.

Priority Assessment transforms configured factors into a score. The implementation should retain factor values and weights so the result can be inspected. Thresholds for labels such as low, medium, high, or critical should be configurable and documented. Missing factors should trigger a clear policy: request the value, apply a documented default, or calculate with a warning. They should not silently be interpreted as safe conditions.

Resource and Candidate Generation compares incident needs with resource capabilities and availability. A resource record may contain a unique identifier, category, capability tags, current location, availability state, capacity, and workload. Candidate generation should retain the reason for inclusion. Constraint Filtering then checks workload, reachability, time windows, and other operational restrictions. When a constraint cannot be evaluated because information is missing, the result should be marked unresolved rather than automatically treated as feasible.

Route Intelligence works on the road or route graph available to the prototype. Nodes represent locations or intersections, and edges represent traversable connections with associated costs. The result may include a path, estimated distance, estimated travel time, and route status. The system should record the graph version and cost assumptions. Allocation combines feasible candidates and route costs to construct a response plan. Explanation turns internal results into operator-readable reasons, while Adaptation stores the previous plan, applies a new event, re-runs the workflow, and summarizes the differences.

XII. DATA MODEL AND INFORMATION FLOW

An incident record should contain an identifier, incident type, severity, affected-population estimate, timestamp, location text, coordinate status, contextual conditions, and current priority score. Original input values should be stored separately from derived values so that a result can be reproduced. Synthetic scenarios should be explicitly labelled.

A resource record should contain an identifier, category, capabilities, availability, location, workload or capacity information, and deployment restrictions. The system should distinguish unavailable resources from resources whose availability is unknown. Exclusive resources should not be assigned to conflicting needs unless the model explicitly allows shared use.

A scenario record defines the starting state and the events applied during a test. A response-plan record should include a version, creation time, incident identifier, selected assignments, unmet needs, route results, feasibility status, and explanation data. Versioning makes before-and-after comparisons reproducible and helps identify which event caused a plan change.

Information should flow from validation to priority calculation, candidate generation, constraint filtering, route estimation, allocation, and explanation. Each stage should return structured outputs rather than display strings alone. This supports unit testing, consistent dashboard rendering, and CSV export. A trace record can store inputs, configuration version, rejected candidates, and decision reasons.

XIII. ALGORITHM OUTLINE AND COMPLEXITY

The main procedure is as follows: validate the incident and resource data; calculate priority; derive required capabilities; generate candidates for each need; apply hard constraints; estimate route costs where possible; rank feasible options; construct an allocation while respecting capacity and assignment conflicts; and produce a response plan with explanations. If an event arrives, save the current plan, update the state, repeat the relevant steps, and calculate the plan difference.

Pseudocode: INPUT incident I, resources R, state S. Validate(I,S); if invalid, return validation errors. ComputePriority(I); Needs = DeriveNeeds(I). For each need, generate candidates using capability and availability. Filter candidates with hard constraints. For each feasible candidate, calculate route status and cost if road data are available. Rank feasible options using configured scoring rules. Construct an allocation that respects capacity and assignment conflicts. If no feasible allocation exists, return a no-feasible-plan result and reasons. Otherwise, return the plan and explanation. When event e arrives, save the current plan, apply e to S, rerun the procedure, and compare plans.

The computational cost depends on the number of incidents, resources, candidate assignments, and route-graph size. A* performance depends on the graph and heuristic and may explore a large part of the reachable network in difficult cases. Candidate generation can be accelerated by indexing resources by capability and availability. Constraint filtering is generally linear in the number of candidates when each check has bounded cost. Allocation can become combinatorial when many needs compete for the same resources; the implementation should define limits, deterministic tie-breaking, and clear handling for infeasible cases.

XIV. TESTING STRATEGY AND SCENARIO MATRIX

Functional tests should verify required fields, supported incident types, valid numeric ranges, unresolved locations, and reproducible priority calculations. Resource tests should confirm that unavailable resources and capability mismatches are excluded. Route tests should use a small graph with a known best path and another graph with no connecting path.

Constraint tests should include an available and capable resource, an unavailable resource, a wrong capability, a resource exceeding its workload limit, and a candidate that cannot reach the incident within the required time window. Each test should define the expected outcome in advance and verify both the feasibility result and the explanation for any rejection.

Adaptation tests should begin with a feasible plan and apply one event, such as marking a resource unavailable or closing a road edge. The system should recompute the plan and identify changed assignments. A second test should apply an event that does not affect the selected resources and check whether the plan remains stable. Multiple-event tests should verify event ordering and reproducibility.

An initial scenario matrix should include a routine incident with plentiful resources, a high-priority incident with scarce resources, two incidents competing for one specialist unit, a road closure, a resource loss after the first plan, missing location data, conflicting time windows, and a case with no feasible allocation. For each scenario, record expected and actual results, execution time, constraint violations, needs covered, and explanation quality. Results must be reported as measured values. Synthetic data must be labelled as such; demonstration success does not establish real-world effectiveness.

XV. ETHICAL, SAFETY, AND OPERATIONAL CONSIDERATIONS

The system is intended to support rather than replace trained emergency personnel. Operators should be able to inspect source inputs, challenge an unsuitable recommendation, and decide whether to use the plan. A computed score should not be presented as an authoritative command. Recommendations should remain subject to verification.

Incident reports, population estimates, resource status, and route data can be incomplete or stale. The interface should expose data source and freshness where available. Unknown values should remain explicitly unknown. If a route uses a static graph, the display should explain that current traffic, closures, and hazards may not be represented.

A weighted score may under-prioritize groups or locations if its factors do not reflect actual need. Priority criteria should be documented, reviewed with stakeholders, and tested across varied scenarios. Authorized overrides should record the reason without silently erasing the original recommendation. Operational records may contain sensitive location and resource information; a deployed system should use access controls, protect credentials, validate inputs, and maintain logs of material decisions and state changes. These are deployment requirements, not claims that every control is already present in the prototype.

XVI. LIMITATIONS AND FUTURE RESEARCH

The weighted priority model depends on its chosen factors and weights. Candidate matching may simplify real-world capability and compatibility. Route estimates may omit live traffic, hazards, and temporary restrictions. A prototype allocation strategy may not capture all command rules or cross-agency agreements. This paper does not report field trials or quantitative comparisons, so operational effectiveness remains unproven.

Future work can investigate uncertainty-aware scoring, multi-incident allocation, richer resource compatibility constraints, and more detailed route costs. A benchmark suite can compare the proposed approach with baseline policies under repeatable conditions. Human-centred evaluation can assess whether explanations help coordinators understand trade-offs and identify data errors. Integration with validated operational data should follow review of permissions, reliability, and governance.

Sensitivity analysis is another useful direction. By varying priority weights, resource availability, and route costs, evaluation can measure how often the chosen allocation changes. Large changes caused by small input variations may indicate a need for better calibration or clearer uncertainty indicators. Such analysis would provide stronger evidence than a small number of successful demonstrations.

XVII. DETAILED OPERATIONAL SCENARIOS

Scenario 1: High-priority incident with adequate resources. The operator enters a severe incident and a location that can be resolved. Priority assessment produces a high score according to the configured weights. Candidate generation identifies capable units, and constraint filtering removes units that are unavailable or exceed their workload limits. Route search estimates the travel cost of the remaining units. Allocation selects a feasible set and the dashboard explains coverage and route status. This scenario checks the basic end-to-end path from input to recommendation.

Scenario 2: Resource shortage. A high-priority incident is entered while only a limited number of resources are available. The system should not imply that all needs can be met if the inventory is insufficient. It should show covered and uncovered needs, identify the resources assigned, and report any feasibility limitation. The operator can then review the shortage and consider external assistance. This scenario tests whether the interface communicates unmet demand rather than hiding it behind a single overall score.

Scenario 3: Route disruption after an initial plan. The system first generates a plan using the available road graph. A simulated road closure then removes an edge or makes it unavailable. The adaptation module updates the graph state and re-runs route search and allocation. The updated plan should show whether the affected resource remains reachable, whether an alternate route exists, and which assignments changed. The comparison should include the reason for each change and should preserve the previous plan for audit.

Scenario 4: Resource becomes unavailable. After the first plan is generated, a selected resource is marked unavailable. The system should exclude it from subsequent candidate generation or feasibility checks and produce a replacement plan if alternatives exist. If no replacement is feasible, the result should clearly state that no feasible assignment was found. This scenario tests the difference between static scheduling and adaptive planning.

Scenario 5: Missing or ambiguous location. If the location cannot be resolved, the system should not calculate a route based on invented coordinates. It should ask the operator to correct the location or mark route-based evaluation as unavailable. Priority assessment may continue if its inputs do not depend on coordinates, but the dashboard should identify the limitation. This scenario tests graceful handling of incomplete information.

XVIII. EXPLANATION DESIGN AND USER INTERACTION

A decision-support interface should explain the recommendation in terms an operator can verify. The summary should show the incident identifier, current priority, required capabilities, number of feasible candidates, selected resources, route status, and remaining unmet needs. It should avoid displaying a precise estimated arrival time when the route or travel-time inputs are missing. When an option is rejected, the interface should name the failed constraint, such as unavailable resource, capability mismatch, workload limit, or no route found.

The explanation should distinguish three types of information. First, input facts are values entered by the user or retrieved from a data source. Second, derived values are calculated by the system, such as the priority score, candidate ranking, or estimated travel time. Third, assumptions are configuration choices, such as factor weights, constraint thresholds, or route-cost rules. Presenting these categories separately helps the operator determine whether an output should be challenged because of bad data, a model assumption, or a computation.

After an event is applied, a plan-difference view should compare the previous and current plan. It can show resources added or removed, changes in route status, newly uncovered needs, and the event that triggered the re-planning. If the new plan is worse under one measure but better under another, the explanation should state the trade-off rather than describing the new plan as universally better. This supports informed human judgement.

XIX. REPRODUCIBILITY AND CONFIGURATION MANAGEMENT

A repeatable experiment requires stable inputs and documented configuration. Each test scenario should have a unique identifier and a stored initial state. Priority weights, thresholds, resource rules, route-cost assumptions, and allocation settings should be versioned. When a plan is generated, the application should retain enough information to reproduce the calculation, subject to data-protection requirements. If the configuration changes, the new version should be identifiable in the resulting record.

Randomness, if used in any simulation, should be controlled through a fixed seed for benchmark runs. The same input should produce the same result unless a documented external input has changed. Where multiple assignments have equal scores, deterministic tie-breaking improves reproducibility. The application should also record the software version and relevant external data version when possible. These practices are useful for debugging, comparing algorithm changes, and preparing a defensible research evaluation.

Reproducibility also requires separating demonstration scenarios from real incident records. Synthetic scenario data should be clearly labelled in the interface and in exported reports. A CSV export should include scenario identifier, plan version, calculation time, selected assignments, route status, unmet needs, and warnings. It should not export sensitive details unnecessarily, and the project should define how exported files are stored and shared.

XX. COMPARISON WITH A SIMPLE BASELINE

A baseline is needed to understand whether the integrated workflow offers benefits beyond a basic selection rule. A simple baseline could select the first available resource matching each need, without route-aware ranking or event-driven plan comparison. A second baseline could rank resources by estimated travel time only. Both baselines should use the same resource inventory and hard constraints where appropriate so that the comparison is fair.

The evaluation should compare feasibility first. A method that produces a shorter estimated travel time but violates a hard constraint should not be considered superior. After infeasible assignments are excluded, the evaluation can compare the proportion of needs covered, total estimated travel cost, number of unassigned needs, computation time, and stability

after events. For adaptive scenarios, a useful measure is whether the system detects and revises an assignment affected by the changed state.

No conclusion about superiority should be drawn until measurements are collected. The expected value of the baseline is methodological: it provides a reference point and reveals whether additional components, such as route search or marginal-value allocation, contribute measurable benefits. If results are mixed, the paper should report the trade-offs instead of selecting only the most favourable metric.

XXI. IMPLEMENTATION MILESTONES

A practical implementation sequence begins with the data model and input validation. The second milestone implements priority scoring and verifies it with hand-calculated test cases. The third milestone adds resource capability matching and hard-constraint filtering. The fourth milestone connects route search to a small known graph and verifies both successful and failed searches. The fifth milestone combines feasible candidates and route costs in the allocation stage. The sixth milestone presents the response plan and explanation through the dashboard. The seventh milestone implements event injection, re-planning, and comparison of plan versions.

Each milestone should include unit tests before it is integrated into the next stage. Integration tests should confirm that invalid data do not pass silently from one module to another and that the output format remains consistent. Interface tests should check that warnings and no-feasible-plan states are visible. Performance testing should be performed only after functional correctness has been established. This staged approach reduces debugging complexity and makes the final system easier to explain during a project review.

REFERENCES

[1] N. Altay and W. G. Green III, “OR/MS research in disaster operations management,” European Journal of Operational Research, vol. 175, no. 1, pp. 475–493, 2006, doi: 10.1016/j.ejor.2005.05.016.

[2] E. Galindo and R. Batta, “Review of recent developments in OR/MS research in disaster operations management,” European Journal of Operational Research, vol. 230, no. 2, pp. 201–211, 2013, doi: 10.1016/j.ejor.2013.01.039.

[3] D. J. Power, Decision Support Systems: Concepts and Resources for Managers. Westport, CT, USA: Quorum Books, 2002.

[4] M. Turoff, M. Chumer, B. Van de Walle, and X. Yao, “The design of a dynamic emergency response management information system (DERMIS),” Journal of Information Technology Theory and Application, vol. 5, no. 4, pp. 1–35, 2004.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[6] A. K. Mackworth, “Consistency in networks of relations,” Artificial Intelligence, vol. 8, no. 1, pp. 99–118, 1977.

[7] R. L. Keeney and H. Raiffa, Decisions with Multiple Objectives: Preferences and Value Tradeoffs. New York, NY, USA: Wiley, 1976.

[8] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[9] Streamlit, “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io/. [Accessed: Oct. 22, 2026].

[10] OpenStreetMap Contributors, “OpenStreetMap.” [Online]. Available: https://www.openstreetmap.org/. [Accessed: Oct. 22, 2026].